



Machine Learning

HOMEWORK 1

NAME:

STUDENT ID#:

Objectives:

- Develop a deeper understanding of **regression and classification** models, focusing on **linear regression and logistic regression**.
- Understand **the mathematical foundations** of regression and classification models, including **least squares estimation** and the **sigmoid (logistic) activation function**.
- Analyze how machine learning models **learn from data using optimization methods** such as **gradient descent**.
- Understand the differences between **continuous prediction problems (regression)** and **categorical prediction problems (classification)**.
- Evaluate model performance using standard metrics such as Mean Squared Error (MSE), confusion matrix, precision, recall, accuracy, ROC curve, and AUC.

Part 1. Background

Regression is a statistical approach used to model the relationship between a dependent variable (output) and one or more independent variables (inputs). Linear regression is commonly used when the relationship between variables can be approximated by a linear function. The model parameters are typically learned by minimizing an error function using optimization methods such as gradient descent.

Classification is another important machine learning task that focuses on predicting categorical labels. Logistic regression is a commonly used classification algorithm for binary classification problems. It uses the sigmoid activation function to convert linear outputs into probabilities, which represent the likelihood of belonging to a particular class.

Part 2. Instruction

1. **Regression.** In this assignment, you are required to implement linear regression and ridge regression models using only NumPy. You will get no points by simply calling `sklearn.linear_model.LinearRegression` or any other built-in regression models from machine learning libraries.
Your task is to train these regression models using **gradient descent** on the provided dataset, evaluate their performance using appropriate regression metrics, and test them on unseen data. You should analyze the model performance and compare the results obtained from different regression approaches.
2. **Classification.** Tasks: In this assignment, you will implement logistic regression using only NumPy. You will get no points by simply calling `sklearn.linear_model.LogisticRegression`. Your task is to train these models on a provided dataset, evaluate their performance, and test to classify them. Compare confusion matrices, ROC curves, and AUC to understand their strengths and limitations.

Lecture: Prof. Hsien-I Lin

TA: Satrio Sanjaya and Muhammad Ahsan



The goal of the classification model is to predict y , which is a binary label (0 or 1). **What are we classifying?** We are predicting whether an individual earns more than \$50K per year based on demographic and work-related factors.

- Input Features: Categorical variables (such as **age_bin**, **occupation_bin**, **education_bin**, etc.), which must be converted into numerical features using one-hot encoding before training.
- Output (Target Variable y): A binary classification label (**0 or 1**).

The dataset and sample code can be found here: <https://github.com/Satriosnjya/ML-Labs.git>

Part 3. Arithmetic Instructions

A. Regression

- | Step | Procedure |
|------|---|
| 1 | <p>Define the Model: Linear regression predicts the output y using a linear function:</p> $y(x, w) = w_0 + w_1 x_1 + \dots + w_D x_D$ <p>In simple linear regression with only one feature x: $y = mx + b$</p> |
| 2 | <p>Mean Square Error (MSE) Loss Calculation</p> <p>From the textbook, Equation (4.11) defines the sum-of-squares error function:</p> $E_D(w) = \frac{1}{2} \sum_{n=1}^N \left(t_n - w^T \phi(x_n) \right)^2$ <p>which directly follows the equation, computing the squared difference between predicted and actual values.</p> |
| 3 | <p>Gradient Descent for Weight Updates</p> <p>From the textbook, Equation (4.12) defines the gradient of the log-likelihood:</p> $\nabla_w \ln p(t X, w, \sigma^2) = \frac{1}{\sigma^2} \sum_{n=1}^N \left(t_n - w^T \phi(x_n) \right) \phi(x_n)^T$ <p>This follows the principle of gradient descent, adjusting weights by subtracting a scaled version of the gradient.</p> |
| 4 | <p>Model Training Using Gradient Descent</p> <p>Equation Reference: From the textbook, Equation (4.22) for sequential learning (LMS Algorithm):</p> $w^{(\tau+1)} = w^{(\tau)} + \eta \left(t_n - w^{(\tau)T} \phi(x_n) \right) \phi_n$ <p>Follows this equation by iteratively updating w based on the gradient.</p> |
| 5 | <p>Mean Square Error (MSE) as Evaluation Metric</p> <p>From the textbook, Equation (4.20) defines the MLE estimate for variance:</p> $\sigma_{ML}^2 = \frac{1}{N} \sum_{n=1}^N \left(t_n - w_{ML}^T \phi(x_n) \right)^2$ |

B. Classification

- | Step | Procedure |
|------|--|
| 1 | <p>Logistic Regression:</p> <ol style="list-style-type: none"> 1. (<u>init</u>) We initialize the weights and bias:
 $w = 0, b = 0$ 2. (<u>fit</u>) <ul style="list-style-type: none"> • Linear Model |



- Sigmoid Function (Logistic Activation Function): $\sigma(a) = \frac{1}{1+\exp(-a)}$
(5.42)

- Update weight : $\nabla E(w) = \sum_{n=1}^N (y_n - t_n) \phi_n$
(5.75)

3. (predict_proba) Uses the sigmoid function to compute probability

4. (predict) Binary Classification Decision

Convert probabilities to class labels using a threshold $\tau = 0.5$

2 Confusion Matrix

A confusion matrix is a table used to evaluate the performance of a classification model. It consists of the following four elements:

$$CM = [TN \ FP \ FN \ TP]$$

where:

- True Positive (TP): The model correctly predicts class 1.
- True Negative (TN): The model correctly predicts class 0.
- False Positive (FP): The model predicts 1 when the actual class is 0.
- False Negative (FN): The model predicts 0 when the actual class is 1.

Several other quantities can be defined as:

$$Accuracy = \frac{N_{TP} + N_{TN}}{N_{TP} + N_{FP} + N_{TN} + N_{FN}} \quad (5.29)$$

$$Precision = \frac{N_{TP}}{N_{TP} + N_{FP}} \quad (5.30)$$

$$Recall = \frac{N_{TP}}{N_{TP} + N_{FN}} \quad (5.31)$$

3 ROC Curve and AUC

The ROC curve is a plot that shows the trade-off between the True Positive Rate (TPR) and the False Positive Rate (FPR) at different classification thresholds. The Area Under the Curve (AUC) for the ROC curve measures the model's ability to distinguish between classes.

True Positive Rate (TPR), Also known as recall or sensitivity:

$$TPR = \frac{TP}{TP + FN}$$

False Positive Rate (FPR), The proportion of incorrectly predicted positives out of all actual negatives:

$$FPR = \frac{FP}{FP + TN}$$

4 F-score

F-score, which is the geometric mean of precision and recall:

$$F = \frac{2 \times precision \times recall}{precision + recall} \quad (5.38)$$

$$= \frac{2 \times N_{TP}}{2N_{TP} + N_{FP} + N_{FN}} \quad (5.39)$$

Part 4. Data Transfer Instructions

Step

Procedure

- 1 Download the dataset and example code from:
<https://github.com/Satriosnjaya/ML-Labs.git>
- 2 Open colab.research.google.com or you may use your **local Python environment**.



- 3 Create a new notebook by clicking “New Notebook.”
- 4 For the Regression task:
 - Upload regression_data.npy
 - Upload Regression_Student.ipynb
 - These files can be found in the “1.Regression folder.”
- 5 For the Classification task:
 - Upload classification.csv
 - Upload Classification_Student.ipynb
 - These files can be found in the “2.Classification folder.”

Grading & Submission Instructions

Your report must present **results, figures, and analysis** based on the implementation tasks. The report should follow the outline below.

1. (40%) Regression

- 1.1. (6%) Implement Standard Linear Regression using Gradient Descent
 - Compute gradients for weight (m) and bias (b).
 - Update weights.
 - Output : Model parameters (weight and bias)
- 1.2. (4%) Evaluate the model using MSE for standard regression
 - Complete compute_mse() function
 - Output : MSE for standard regression
- 1.3. (6%) Implement Ridge Regression with L2 Regularization
 - Modify loss function to include L2
 - Compute updated gradients for weight and bias.
 - Output : Model parameters and MSE for Ridge Regression
- 1.4. (6%) Plot the training loss curve.
 - Store the loss at each iteration and plot it using matplotlib.
 - Try different values for: Learning Rate (lr) and Number of Iterations ($iterations$)
 - Output : Loss curve comparison of Standard regression and ridge regression
- 1.5. (6%) Implement Closed-form Ridge Regression (Refer to Equation 4.27)
 - Fill in closed_form_ridge() function
 - Compute w_{closed_form}
 - Output : Model parameters and MSE for closed-form Ridge Regression
- 1.6. (6%) Implement predictive distribution (Refer to Equation 4.33)
 - Fill in predictive_mean and predictive_variance
 - Print 5 values of predictive_mean
 - Output : Predictive variance & example predictions
- 1.7. (6%) Visualize predictions and confidence intervals
 - Fill in missing values in scatter plot (plt.scatter)
 - Implement confidence interval shading using plt.fill_between().
 - Output :
 - 1) Prediction plot of standard regression, ridge regression and closed-form regression.
 - 2) Predictive distribution with confidence interval.



2. (40%) Classification

2.1. (12%) Implement your Logistic Regression model from scratch

Train the model using

- learning rate = 0.001, and iteration = 2000
- learning rate = 0.00001, and iteration = 6000
- learning rate = 0.0001, and iteration = 1500

2.2. (14%) Compute Classifier Accuracy

- Compute True Positive (TP), True Negative (TN), False Positive (FP), and False Negative (FN).
- Construct a confusion matrix heatmap for logistic regression.
- Compute the evaluation for Precision, Recall, and Accuracy.

2.3. (14%) Generate and Analyze the ROC Curve.

- Compute TPR and FPR for different threshold.
- Plot the ROC curve and show the AUC score.
- Compute the F-score.

3. (20%) Question

- 3.1. (5%) Based on your confusion matrix, analyze the types of errors made by the model. Identify which type of error (False Positives or False Negatives) occurs more frequently in your dataset. Explain why this might happen.
- 3.2. (5%) Discuss the impact of **different learning rates** and **iterations** on the convergence of logistic regression. How does hyperparameter tuning affect performance? *Include and compare the ROC curves for different hyperparameter settings.
- 3.3. (5%) Discuss the sigmoid activation function used in logistic regression. How does the activation function influence the decision boundary of the model?
- 3.4. (5%) Define and explain the significance of Confusion Matrix, ROC Curves, and AUC in evaluating classification models. How do they contribute to model selection?

Submission:

1. **Report:** Include screenshots of your results for each task (model training, evaluation, and plots).
2. **Code:** Submit your complete Python script (.py or .ipynb notebook).
3. Upload both your report and code to the E3 system. Name your files correctly as StudentID_HW1.zip with the following directory structure:

```
StudentID_HW1.zip |
├── Regression
│   ├── Report: StudentID_report.pdf
│   ├── Code: StudentID_code.py or StudentID_code.ipynb
│   └── Dataset: regression_data.npy |
└── Classification
    ├── Report: StudentID_report.pdf
    ├── Code: StudentID_code.py or StudentID_code.ipynb
    └── Dataset: classification.csv
```

4. Submission Rules:
 - 1 day late: **10% deduction** from the total score.
 - Incorrect file name: **10% deduction**.
 - Incorrect directory structure: **10% deduction**.
5. Plagiarism is **strictly prohibited**. Copying reports or code from **other students, previous submissions, or any external sources will result in penalties**.



Sample Output: Please note that it is for reference only

